Laboratory Guide 3: Quantum Algorithms

Oracles, Phase Kickback, and the Quantum Advantage

Course: Quantum Computing Foundation

1 Introduction and Objectives

In the previous labs, we learned how to manipulate qubits and create entanglement. Now, we transition from physics to computer science. We will implement **Deutsch's Algorithm**, a milestone in quantum computing. It is the simplest algorithm that unequivocally proves a quantum computer can solve a specific problem faster than any classical machine.

Prerequisites:

- Lesson QC12: *Algorithms - Quantum Parallelism*
- Lesson QC13: *Deutsch's Algorithm*
- Lab 2: *Multi-Qubit Systems*

Learning Objectives:

1. Understand the Query Model and the role of a Quantum Oracle.
2. Visualize and master the "Phase Kickback" mechanism using the Composer.
3. Track the mathematical evolution of the quantum state step-by-step.
4. Implement Deutsch's Algorithm in Qiskit using professional coding standards.

Important: The Black Box (Oracle)

What is an Oracle?
In computational complexity, an Oracle is a "Black Box". Suppose we have a function $f(x)$. We can give the Oracle an input x, and it returns the output $f(x)$. Crucially, we **cannot** look inside the box to see the circuitry or the code. Our goal is to discover global properties of $f(x)$ while minimizing the number of queries (questions) asked to the Oracle.

2 Part A: The Secret Mechanism (IBM Quantum Composer)

Platform: IBM Quantum Composer

2.1 Exercise 1: Phase Kickback

Context: As explained in Lesson **QC13**, quantum parallelism allows us to evaluate $f(0)$ and $f(1)$ simultaneously. However, measuring the state directly destroys the superposition and only gives us one answer. To extract the relation between $f(0)$ and $f(1)$, we rely on a physical phenomenon called **Phase Kickback**.

When a target qubit is in the superposition state $|-\rangle$, applying a controlled operation (like a CNOT) does not flip the target's probabilities. Instead, the phase change "kicks back" and alters the phase of the *control* qubit.

Student Activity

Action:

1. Add an `H` gate to q_0 (Control). It is now in state $|+\rangle$.
2. Add an `X` gate, then an `H` gate to q_1 (Target). It is now in state $|-\rangle$.
3. Apply a `CNOT` gate (Control q_0, Target q_1).
4. **Look at the Q-Sphere** before and after the CNOT.

Expected Effect: Normally, a CNOT flips the target. Here, the target q_1 remains strictly $|-\rangle$. Instead, the phase of the control qubit q_0 flips from $|+\rangle$ to $|-\rangle$! The "action" was transferred backward to the control.

IBM Composer View (Q-Sphere)

Q-Sphere Visualization
Before the CNOT, the dots on the Q-Sphere for the control qubit are Blue (Positive phase). After the CNOT, the dot corresponding to the $|1\rangle$ component of the control qubit turns **Red** (Negative phase). The target qubit's probabilities and phases did not change at all. This Red dot is the core of quantum computation.

3 Part B: The Algorithm Step-by-Step

3.1 Exercise 2: Finding the Function Type

The Problem: We are given a function $f(x)$ that takes 1 bit (0 or 1) as input and returns 1 bit. There are two possible categories for this function:

- **Constant:** Returns the same value for all inputs (i.e., $f(0) = f(1)$).
- **Balanced:** Returns different values (i.e., $f(0) \neq f(1)$).

Classically, you must evaluate the function twice ($x = 0$ and $x = 1$) to compare the outputs. Deutsch's Algorithm solves this with **exactly 1 quantum query**.

Step-by-Step State Evolution

How it works: The Mathematical Evolution
Let's trace the state $|\psi\rangle$ of our two qubits (Input x, Ancilla y) through the algorithm.

1. **Initialization:** We start with the input at $|0\rangle$ and the ancilla at $|1\rangle$.

$$|\psi_0\rangle = |0\rangle \otimes |1\rangle$$

2. **Superposition (Apply H to both):** We prepare the input to query both values simultaneously, and the ancilla to absorb the phase kickback.

$$|\psi_1\rangle = |+\rangle \otimes |-\rangle = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)\left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right)$$

3. **The Oracle Query U_f:** The quantum oracle performs the operation $|x\rangle|y\rangle \to |x\rangle|y \oplus f(x)\rangle$. Because the ancilla is in the $|-\rangle$ state, the function's output $f(x)$ is transferred to the phase of the input qubit:

$$|\psi_2\rangle = \left(\frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}}\right)|-\rangle$$

4. **Interference (Apply H to Input):** We apply a final Hadamard gate to the input qubit to extract the phase difference.
   - If $f(0) = f(1)$ (Constant), the phases are the same, and the state interferes constructively back to $\pm|0\rangle$.
   - If $f(0) \neq f(1)$ (Balanced), the phases are opposite, and the state interferes constructively to $\pm|1\rangle$.

Conclusion: A measurement of 0 means Constant. A measurement of 1 means Balanced.

3.2 Exercise 3: Coding Deutsch's Algorithm in Qiskit

We will now implement the algorithm described above. We will use a CNOT gate as our Oracle. A CNOT acts as $f(x) = x$, which is a **Balanced** function.

Coding Task: Deutsch's Algorithm Implementation

```python
"""
Deutsch's Algorithm Implementation.
This script demonstrates the algorithm using a balanced oracle.
"""
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

def run_deutsch_algorithm() -> dict:
    """
    Builds and executes Deutsch's algorithm for a balanced function f(x) = x.

    Returns:
        dict: The measurement counts, proving the function type.
    """
    # 1. Setup: 1 Input Qubit (q0), 1 Ancilla Qubit (q1), 1 Classical Bit
    qc = QuantumCircuit(2, 1)

    # 2. State Preparation (Superposition and Phase Target)
    qc.h(0)            # Input to |+>
    qc.x(1)            # Ancilla to |1>
    qc.h(1)            # Ancilla to |-> for phase kickback
    qc.barrier()

    # 3. The Oracle: Balanced function (CNOT)
    # This represents the unknown black box.
    qc.cx(0, 1)
    qc.barrier()

    # 4. Interference and Measurement
    qc.h(0)            # Extract the relative phase
    qc.measure(0, 0)   # Read the result into the classical bit

    # 5. Execution
    simulator = AerSimulator()
    job = simulator.run(qc, shots=1000)
    return job.result().get_counts()

# Run the simulation and print the results
counts = run_deutsch_algorithm()
print(f"Algorithm Output: {counts}")
```

Analysis: Because the oracle we built is balanced, the output will be exactly ''1': 1000'. If you replaced the Oracle block with an empty space (Identity gate), it would act as $f(x) = 0$ (Constant), and the output would be ''0': 1000'.

Deep Dive: Beyond Deutsch (QC14)

Beyond Deutsch: Why does this matter? (Lesson QC14)
Deutsch's algorithm has no direct commercial application. However, it proves that quantum interference can solve a problem in $\mathcal{O}(1)$ steps instead of $\mathcal{O}(N)$.

- **Deutsch-Jozsa:** Expands this to N bits. A classical computer needs $2^{N-1} + 1$ queries in the worst case. The quantum computer still needs exactly **1 query**. This represents an exponential speedup.
- **Grover & Shor:** The logic of Oracles and Phase Kickback is the exact same engine used by Grover's algorithm to search databases and Shor's algorithm to break RSA encryption.

4 Part C: Assessment (Quiz)

Q1. The Power of the Oracle
In the Query Model of computation (QC12), what is the primary metric used to evaluate the efficiency of a quantum algorithm?

A) The clock speed of the quantum processor.

B) The physical number of qubits required.

C) The number of times the algorithm evaluates the Oracle function $f(x)$.

Q2. The Core Mechanism
In the mathematical evolution of Deutsch's algorithm, what causes the value of $f(x)$ to be encoded into the *phase* of the input qubit?

A) The entanglement between the two qubits.

B) The Phase Kickback effect, which occurs because the ancilla qubit is initialized in the $|-\rangle$ state.

C) The final measurement operation collapsing the wavefunction.

Q3. Reading the Output
You run Deutsch's algorithm on a mysterious Oracle and measure the input qubit at the very end. The result is 100% certainty of reading **"0"**. According to the interference step, what did you just prove about the hidden function?

A) The function is Balanced (e.g., $f(0) \neq f(1)$).

B) The function is Constant (e.g., $f(0) = f(1)$).

C) The Oracle contains a hardware error.

Solutions: Q1: C (As stated in QC12); Q2: B (The minus sign is required to kick back the phase); Q3: B (Constructive interference on $|0\rangle$).